

Introduction to Final Assignment

COMSM0086

Dr Simon Lock and Dr Sion Hannuna

Simple Text Adventure Game (STAG)

Overview

Final exercise is worth 70% of unit assessment

The aim of this assignment is to build...

A general-purpose socket-server game-engine
(for text adventure games)

If you are unfamiliar with the genre, take a look at:
<https://tinyurl.com/zork-game>

Game Server

The main class is a server that listens on port 8888
Server accepts incoming commands from clients
It then processes command & changes game state
Returning a suitable response back to the client

After processing command server closes connection
Then listens for the next connection on port 8888
Don't panic: this is provided for you in the template

Standard "Built-in" Gameplay Commands

- "inventory" (or "inv" for short): lists all of the artefacts currently in the possession of the player
- "get": picks up a specified artefact from current location and adds it to the player's inventory
- "drop": puts down an artefact from player's inventory and places it into the current location
- "goto": moves the player to a new location (only if there is a valid path to that location)
- "look": describes the current location, including all entities in that location and paths to other locations

Demo of Server in Action

RunGameServer

In order to connect to the server
We have also provided you with a GameClient:

RunGameClient

You won't need to alter the client code !
It is really just there so you can play the game

With this server, you can play ANY game !

But how is this possible ?

Gameplay is defined by providing two separate 'game configuration' files to the game engine:

- Entities: structural layout and relationships
- Actions: dynamic behaviours of the game

Before considering the content of each of these files
Let us discuss 'Entities' and 'Actions' at high level...

Different Types of Entity

- Character: A creature/person involved in game
- Player: A special kind of character (the user !)
- Location: A room or place within the game
- Artefact: A physical "thing" within the game
(these things CAN be collected by the player)
- Furniture: A physical "thing", part of a location
(these things CANNOT be collected by the player)

The Location Class

Locations are complex entities in their own right
They can contain the following optional elements:

- Paths to other Locations (these can be one-way!)
- Characters that are currently at that Location
- Artefacts currently present in that Location
- Furniture that belongs in the Location

Actions

Dynamic behaviours are represented as 'Actions'
Each action may have the following elements:

- A set of possible 'trigger' key phrases
(ANY of which can be used to initiate an action)
- A set of 'subject' entities that must be available
(ALL of which be present to perform the action)
- A set of 'consumed' entities that are removed
(ALL of which are "eaten up" by the action)
- A set of 'produced' entities that are created
(ALL of which are "generated" by the action)

Example Actions

The previous description of 'Actions' is probably a little hard to understand !

Let's look at some examples by way of illustration
They are represented using a language called XML
(the same language used by the Maven POM file !)

`actions.xml`

Entity Files

Rather than also representing entities using XML
We will use an alternative language called 'DOT'
(It's good to experience a range of languages)

DOT is a language used for representing graphs
(which is basically what a text adventure game is !)

`entities.dot`

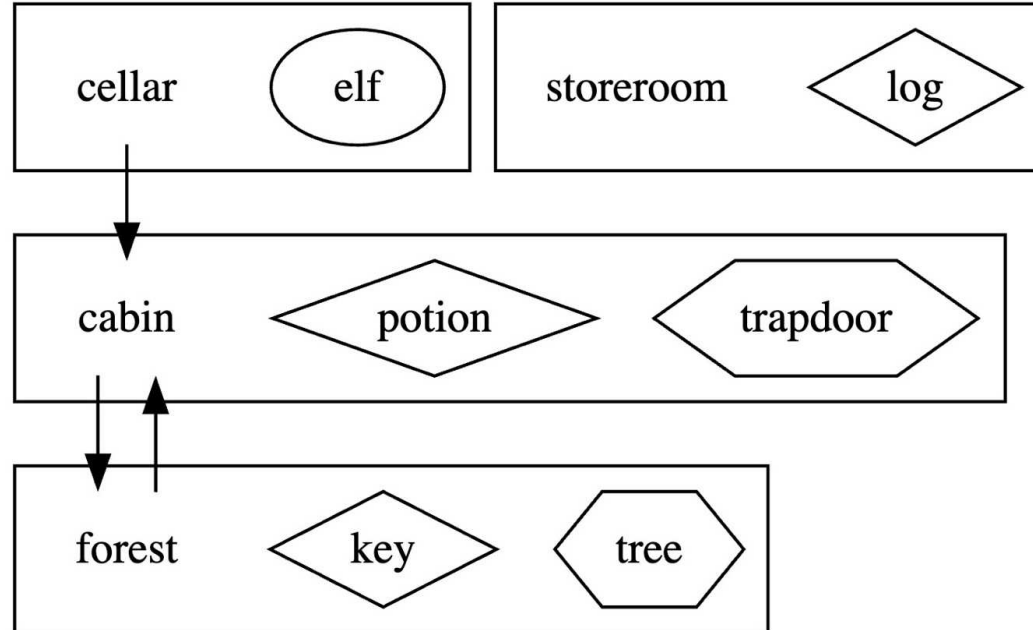
Sorry about the file extension - it's not my choice !

Visualising DOT files

The BIG bonus of using DOT files is that...

There are tools available for rendering them (GraphViz)

We can SEE structure of the 'entities' within the game:



Online Editor and Visualiser

<https://dreampuf.github.io/GraphvizOnline/>

20%20%20%20%20%20tree%20%5Bdescription%20%3D

Parsers

You've already gained experience writing parsers

We don't really want to "go over old ground"

So we will be using two existing parsing libraries !

There is considerable educational value in
learning to use existing libraries and frameworks

This can get lost in a desire to learn fundamentals
But not on this unit !

Which parsers ?

For parsing DOT files, you should use 'JPGD':

<http://www.alexander-merz.com/graphviz/doc.html>

Library `_should_` already be embedded in maven project

And the Java API for XML Processing 'JAXP':

<http://oracle.com/java/technologies/jaxp-introduction.html>

Core library that `_should_` be available to your project

See "test" folder for examples of these parsers in use

Command Interpreter Flexibility

Just as with the DB server, the STAG server must be able to cope with various command variabilities:

- Varying case: all command are case-insensitive
- Varying word order: triggers/subjects in any order
 - unlock door with key
 - use key to unlock door
- Decorated commands: extra "unnecessary" words
 - please unlock the door using the key, thanks !
- Partial commands: some subjects not mentioned
 - unlock door

Full details of all these are contained in workbook

Correctness and Certainty

Your server should block "badly formed" commands

Extraneous Entities

Stated by user, but not defined as a subject of action

open door with key and axe

Ambiguous Commands

Command matches more than one possible action

open with key

(if there are two things that can be opened with a key)

Player Identification

Incoming commands always begin with a username
(to identify which player has issued that command)

A typical incoming message might take the form of:

`simon: open door with key`

This allows the game to support multiple players !

Server does NOT need to deal with authentication
(that would add to the complexity of assignment)

Marking Process

A special set of "custom" game configuration files will be used to assess your engine during marking.

It is therefore essential your code is able to load in files in the same format as the examples provided !

Test scripts will be used to automatically test your game engine to ensure it is operating correctly.

It is therefore essential that you adhere to the "built-in" commands detailed previously !

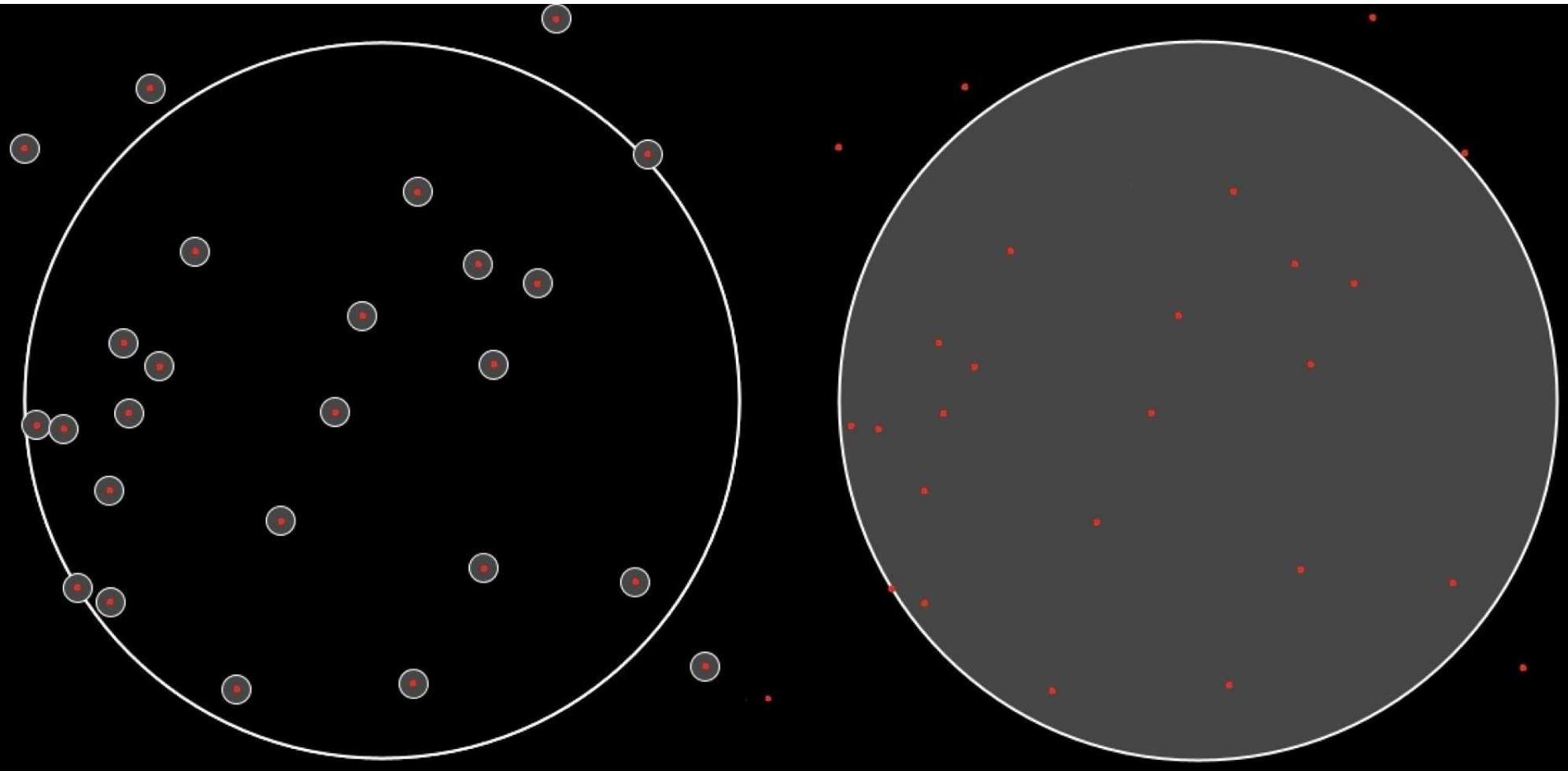
Your own test cases

As we have discussed previously on this unit
Effective and comprehensive testing is essential !
It becomes EVEN more important with AI code gen
(so we can be sure that generated code is correct)

For this reason, YOUR test cases will be marked:

- The extent to which tests cover required features
- Checking system does not enter erroneous states

How do we identify "poor" testing ?



Assessing your test cases (3 ways)

We first ensure that your tests cover all YOUR code
(Whitebox testing ensuring all your code is "exercised")

How do we know that you have written "good" tests ?
(not just a set of "easy" cases that YOUR code passes)

We'll run YOUR tests against a "known good" solution
(Blackbox testing another solution - UNSEEN codebase)

Remember "a successful test is one that finds a fault" ?

We also run your tests against a "known bad" solution
To see how many faults YOUR tests are able to detect !

Code Quality

Your code quality will be assessed during marking
Adhere to guidelines outlined in the quality lecture
Refer to any feedback you have received previously
(if you submitted OXO or DB for code quality analysis)

Last year approx a third of students received a penalty
For those that did, the average penalty was only 2%
Our aim here is not really to penalise students
But rather to get you to improve your code quality !

Collusion

This is an individual assignment, not group activity
Automated checkers used to flag possible collusion
If markers feel collusion has indeed taken place...
Incident is referred to academic malpractice panel

May result in a mark of zero for assignment
or even the entire unit (if it is a repeat offence)

Derived Code

We'll use analysis tools to determine "derived" code
Material that was "found" online or generated by AI

Any derived code will be discounted during marking
You'll only receive credit for code YOU have written

More detail on derivation detection ?

Mashup Coursework Submissions

The notion of a "mashup" codebase is nothing new
It does not rely on availability of effective AI tools
Students have been creating mashup work for years
borrow a bit of code from here, borrow a bit from there

Modern AI tools just make process quicker & easier
Students often don't even realise they are doing it !
But that's what you get when you Gen AI some code

Addressing this Problem

Aim of unit is to assess YOUR programming skills

We aren't going to be able to do that if you are:

- "Reusing" bits and pieces from online solutions
- "Generating" chunks of code using AI tools

If we could **detect** code that has been "derived"...

We could factor it out during the marking process:

you don't get marks for code you didn't write

This would allow us to give credit where credit's due

Implementing this Approach

We have a library of 500 solutions to the assignment
ALL submissions of ALL students from previous years

Many are available online (as part of code portfolios)
These will be part of the training set used by AI tools

These 500 solutions also borrow from online examples
(which provides us with even broader sampling)

All we do is compare YOUR code with this training set

The more similarities that are found...

The more "derivative" your submission has become

I know what you are thinking...

There is BOUND to be some level of similarity
Some standard boilerplate code that everyone has
Or some convergent algorithm that everyone uses

That's fine, we expect this kind of thing to happen
We will be using a variety of filters and thresholds
The aim being to factor out "natural" commonality

Also human-in-the-loop to catch any false-positives

REMEMBER

You will only be rewarded for code that YOU write
Any "derived" code ("found" items or AI generated)
Will be discounted during the marking process
(resulting in a reduced final mark)